4-bit Array Multiplier from RTL to GDSII

with Open-Source Tools on SkyWater SKY130
(`sky130_fd_sc_hd`)

Digital VLSI Design – Team Assignment Report

Name	Roll Number
Krishna Goel	2023112009
Dhimant Bhuva	2023102063

GitHub Repository:

https://github.com/krishna2882/RTL_to_GDS_assignment

Academic Year 2026

Contents

1 Part A: RTL Design and Verification **2**
1.1 Design . . . 2
1.2 Hand-Worked Example (Task 5) . . . 2
1.3 Waveform and Latency . . . 3
1.4 Verification Results . . . 3

2 Part B: Logic Synthesis with Yosys **3**
2.1 Synthesis Summary . . . 4
2.2 Full Cell List (`synth/mult_array_stat.rpt`) . . . 4

3 Part C: Static Timing Analysis with OpenSTA **4**
3.1 STA Summary . . . 4
3.2 Critical-Path Cell Breakdown . . . 5
3.3 Comparison with $2N$ . . . 5

4 Part D: Physical Design with LibreLane **5**
4.1 Iterative Run Log . . . 5
4.2 Final PnR Summary . . . 6
4.3 Cells Added by LibreLane . . . 6
4.4 Layout Screenshots . . . 6

5 Answers to the Questions **7**

6 Team Member Contributions **8**

Part A: RTL Design and Verification

Design

`rtl/full_adder.v` implements the full adder with two `assign` statements:

$$\text{sum} = a \oplus b \oplus \text{cin}, \qquad \text{cout} = ab + \text{cin}(a \oplus b).$$

`rtl/mult_array.v` registers `a` and `b` into 4-bit registers `a_r` and `b_r`, forms the 16 partial-product bits $\text{pp}_i[j] =$ `a_r`$[j]$ AND `b_r`$[i]$, adds them in three rows of four `full_adder` instances (12 in total) exactly as in the Section 2 connection table, and registers the 8-bit result into `p`. All 16 flip-flops have an asynchronous, active-low reset (`rst_n`) that clears them to 0. The `*` operator is not used.

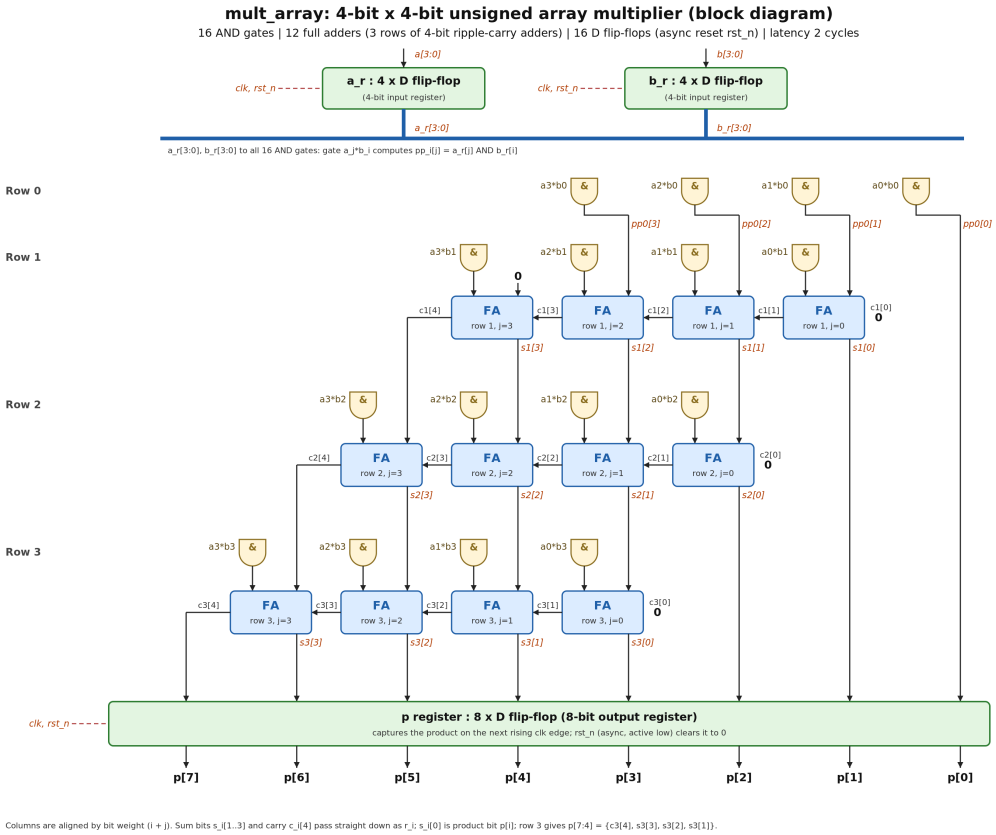


Figure 1: Block diagram: input registers `a_r` and `b_r` (8 flip-flops), 16 AND gates, 12 full adders in three ripple-carry rows, and the 8-bit output register `p`. Columns are aligned by bit weight $i + j$.

Hand-Worked Example (Task 5)

Roll number **2023112009** (Krishna Goel) was used: last digit 9, so $A = 9 + 5 = 14$; $A \neq 7$, so $B = 7$.

```
          1 1 1 0        A = 14
        x 0 1 1 1        B = 7
---------------
          1 1 1 0        pp0 = A AND b0  (b0 = 1)
        1 1 1 0          pp1 = A AND b1  (b1 = 1), shifted 1
      1 1 1 0            pp2 = A AND b2  (b2 = 1), shifted 2
    0 0 0 0              pp3 = A AND b3  (b3 = 0), shifted 3
---------------
0 1 1 0 0 0 1 0        P = 98    (14 + 28 + 56 + 0)
```

Row	Addition	Result	Product bit	Passed down
0	–	pp0 = 1110	p[0] = 0	r0 = 0111
1	pp1 + r0 = 1110 + 0111	s1 = 0101, c1[4]=1	p[1] = 1	r1 = 1010
2	pp2 + r1 = 1110 + 1010	s2 = 1000, c2[4]=1	p[2] = 0	r2 = 1100
3	pp3 + r2 = 0000 + 1100	s3 = 1100, c3[4]=0	p[3] = 0	p[7:4] = 0110

Result: `p = 0110 0010 = 98`. The same intermediate values (`r0=0111`, `s1=0101`, `r1=1010`, `s2=1000`, `r2=1100`, `s3=1100`) were read from the simulated design.

Waveform and Latency

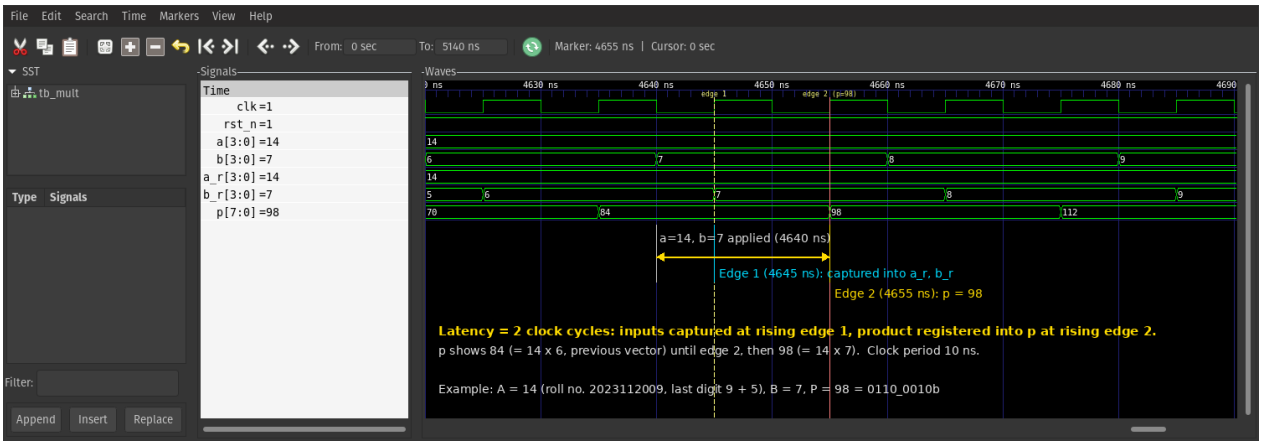


Figure 2: GTKWave view of `mult.vcd` for $A = 14$, $B = 7$. Inputs change at 4640 ns, are captured into `a_r` and `b_r` at rising edge 1 (4645 ns), and $p = 98$ appears at rising edge 2 (4655 ns): a latency of **2 clock cycles**. Before edge 2, `p` still shows $84 = 14 \times 6$ from the previous vector.

Verification Results

The testbench `tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first 2 cycles, applies all 256 (a, b) combinations, waits the 2-cycle latency, compares `p` with $a \times b$, prints a line for every mismatch and one summary line, and writes `mult.vcd`.

Run	Summary line printed
RTL simulation (task 4)	`PASS: 256 vectors checked, 0 errors`
Error test: `sum = a^b` (task 6)	`FAIL: 256 vectors checked, 95 errors`
	first: `MISMATCH: a=3 b=3 expected=9 got=5 (time 1060 ns)`
After undoing the change	`PASS: 256 vectors checked, 0 errors`

The error test fails exactly the 95 input pairs in which at least one full adder receives a carry-in of 1, because the faulty sum ignores `cin`; this was confirmed with a bit-level model of the array.

Part B: Logic Synthesis with Yosys

`synth/synth.tcl` runs the seven required Yosys steps (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat`, `write_verilog`) with the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, via `yosys -c synth/synth.tcl`.

Synthesis Summary

Metric	Value
Total number of cells	67
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	784.502
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	51.04 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (9), `xnor2_1` (7)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Full Cell List (`synth/mult_array_stat.rpt`)

Cell	Count	Area (μm^2)	Cell	Count	Area (μm^2)
`dfrtp_1`	16	400.384	`lpflow_isobufsrc_1`	2	12.512
`nand2_1`	9	33.782	`mux2i_1`	2	20.019
`xnor2_1`	7	61.309	`nor2_1`	2	7.507
`maj3_1`	5	50.048	`o21ai_0`	2	10.010
`and2_0`	4	25.024	`a2111oi_0`	1	8.758
`a21oi_1`	3	15.014	`nand3_1`	1	5.005
`a22oi_1`	3	22.522	`nand4_1`	1	6.256
`xnor3_1`	2	45.043	`nor2b_1`	1	6.256
`and4_1`	2	17.517	`or3_1`	1	6.256
`clkinv_1`	2	7.507	`xor3_1`	1	23.773

The gate-level simulation used the command from the assignment with the SKY130 cell models (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and the same testbench; it passed all 256 vectors, so synthesis did not change the behaviour. The netlist is flattened (no `full_adder` module remains).

Part C: Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical-corner library and the synthesized netlist, creates `clk` with period T, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path. With $T = 10$ ns the worst slack is 7.020 ns, so:

$$T_{\min} = 10 - 7.020 = 2.980 \text{ ns}.$$

Re-running with $T = 2.98$ ns gives a slack of 0.000 ns (MET); $T = 2.97$ ns gives -0.01 ns (VIOLATED).

STA Summary

Metric	Value
Worst slack at $T = 10$ ns	7.020 ns
$T_{\min}$	2.980 ns
$F_{\max}$	$1000/2.980 = 335.57$ MHz
Slack at $T = T_{\min}$	0.000 ns
Critical path start point	`_106_`: `b_r[0]` (dfrtp_1, CLK pin)
Critical path end point	`_100_`: `p[6]` (dfrtp_1, D pin)
Cells on critical path	10: launch FF + 8 logic gates + capture FF
Latency ($= 2 \times T_{\min}$)	5.96 ns
Throughput	335.57 million results/s

Critical-Path Cell Breakdown

Cells on the critical path, in order (from `sta/mult_array_timing.rpt`). The function of each gate was found by simulating the netlist and the RTL side by side for all 256 inputs.

#	Cell	Type	Delay (ns)	Arrival (ns)	Function in the array
1	_106_	dfrtp_1	0.411	0.411	b_r[0] register (clk-to-Q)
2	_048_	and4_1	0.218	0.629	partial products + row 1 carry (merged)
3	_050_	a2111oi_0	0.428	1.057	row 1 carry logic (merged)
4	_051_	a21oi_1	0.094	1.151	row 1 carry logic (merged)
5	_053_	mux2i_1	0.218	1.369	NOT s1[3]: sum of FA row 1, $j = 3$
6	_069_	xnor3_1	0.374	1.742	s2[2]: sum of FA row 2, $j = 2$
7	_074_	maj3_1	0.371	2.113	c3[2]: carry of FA row 3, $j = 1$
8	_076_	maj3_1	0.376	2.489	c3[3]: carry of FA row 3, $j = 2$
9	_080_	xor3_1	0.355	2.844	s3[3]: sum of FA row 3, $j = 3$
10	_100_	dfrtp_1	setup 0.136	–	p[6] register

Comparison with $2N$

All 8 logic gates implement adder-array logic; 5 of them (gates 5–9) compute a specific full-adder output, while gates 2–4 are row 1's partial-product AND gates and carry ripple merged by ABC. In RTL terms the path traverses **8 full-adder stages**:

$$\underbrace{\mathrm{FA}(1,0)\text{–}\mathrm{FA}(1,3)}_{\text{row 1 ripple}} \;\rightarrow\; \mathrm{FA}(2,2) \;\rightarrow\; \underbrace{\mathrm{FA}(3,1)\text{–}\mathrm{FA}(3,3)}_{\text{row 3 ripple}}$$

This equals the estimate $2N = 2 \times 4 = 8$; a depth count over the Section 2 table confirms that p[6] and p[7] are the deepest bits. The logic delay is $2.844 - 0.411 = 2.433$ ns, about 0.30 ns per full-adder stage.

Part D: Physical Design with LibreLane

pnr/config.json contains DESIGN_NAME, VERILOG_FILES, CLOCK_PORT, CLOCK_PERIOD, FP_SIZING = "absolute" and DIE_AREA = [0, 0, 150, 150]. The first CLOCK_PERIOD was $1.2 \times T_{\min} = 3.576$ ns, rounded up to 4.0 ns. LibreLane signs off setup timing in every process corner; every failing run violated only in the slow corners (ss_100C_1v60), so CLOCK_PERIOD was increased by 0.5 ns per run until all checks passed.

Iterative Run Log

Run	Period (ns)	Worst-corner slack (ns)	tt-corner slack (ns)	DRC	LVS	Result
1	4.0	−1.953	0.883	0	0	Fail (ss setup)
2	4.5	−1.453	1.383	0	0	Fail (ss setup)
3	5.0	−0.953	1.883	0	0	Fail (ss setup)
4	5.5	−0.456	2.382	0	0	Fail (ss setup)
5	6.0	−0.128	2.818	0	0	Fail (ss setup)
6	6.5	−0.157	3.089	0	0	Fail (ss setup)
7	7.0	+0.089	3.528	0	0	**Pass**

Run 6 was slightly worse than run 5 because the timing-repair step inserted fewer setup buffers as the period grew (13 in runs 1–4, 11 in run 5, 2 in run 6, 1 in run 7), so the slow-corner path got slower.

Final PnR Summary

Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	7
Core area (μm^2)	17,759.5 (die 150×150 = 22,500)
Cells after synthesis (Part B)	67
Cells after place and route	1,901
Kinds of cells added	decap 1305, fill 260, tap 245, buf 20, clkbuf 3
Wirelength (µm)	2,348
Setup slack (worst corner `ss_100C_1v60`)	+0.089 ns
Setup slack (`tt` corner)	+3.528 ns
Post-route F_{max}	$1000/(7.0 - 0.089) = 144.70$ MHz
F_{max} from Part C	335.57 MHz
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0

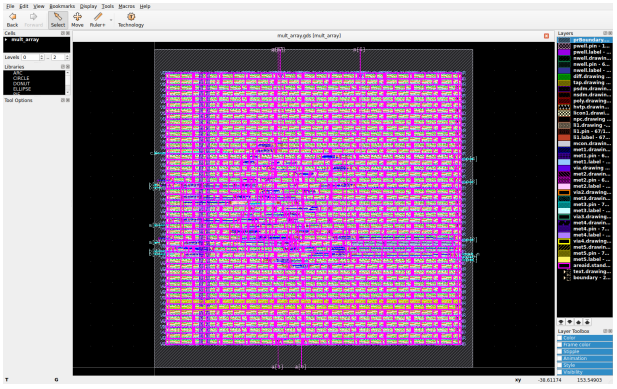
Cells Added by LibreLane

1901 = 16 FF + 52 logic (LibreLane re-synthesises) + 1833 physical cells:

Kind	Cells (from final netlist)	Count
Decap	`decap_12`×900, `decap_6`×193, `decap_3`×125, `decap_8`×50, `decap_4`×37	1305
Fill	`fill_1`×218, `fill_2`×42	260
Tap	`tapvpwrvgnd_1`	245
Timing/repair buffers	`buf_2`×17, `buf_4`×1, `clkdlybuf4s50_1`×2	20
Clock-tree buffers	`clkbuf_16`×3	3

The final run also reports 11 max-slew warnings in the slow corners, all on the reset net `rst_n` (one buffer drives the `RESET_B` pins of 10 flip-flops: 0.781 ns against a 0.75 ns limit). They are *warnings*, not errors; the typical corner has none. The final post-layout netlist was also simulated with the testbench and passed all 256 vectors.

Layout Screenshots



(a) Whole die (150 µm × 150 µm): core and I/O pins



(b) Zoomed view (≈ 36 µm × 22 µm): rows and routing

Figure 3: Final layout (`pnr/mult_array.gds`) in KLayout: (a) entire 150 µm × 150 µm die with standard-cell rows, power grid, and I/O pins; (b) zoomed view near die centre showing cell rows with horizontal power rails, metal-1/2 routing, and vias.

Answers to the Questions

Q1. How many cells did synthesis produce for the logic, and why is it not $16 + 12$?

Synthesis produced **67 cells** in total, of which 16 are D flip-flops (`dfrtp_1`), leaving **51 cells** for the combinational logic. This count is 51 rather than $16 + 12 = 28$ due to three main reasons:

1. **Standard Cell Decomposition.** The `sky130_fd_sc_hd` library has no single full-adder macro; each adder is split into separate library gates for sum (XOR/XNOR) and carry (majority `maj3`, compound AOI/OAI `a21oi`/`a22oi`/`a2111oi`, and MUX-inverters `mux2i`).
2. **Logic Optimisation and Gate Merging.** The synthesis engine (ABC) flattens the hierarchy and merges the 16 partial-product AND gates directly into downstream adder logic using multi-input compound gates.
3. **Constant Propagation.** Adders receiving constant zeroes ($c_i[0] = 0$ and $r_0[3] = 0$) are simplified to half adders or buffers, pruning redundant gates.

Because logic is globally decomposed, merged, and pruned, there is no 1-to-1 mapping between RTL blocks and synthesised cells.

Q2. What is the critical path, how many cells are on it, and how does it compare with $2N$?

The critical path runs from input register `b_r[0]` (`_106_`, clock-to-Q 0.411 ns) to output register `p[6]` (`_100_`, setup time 0.136 ns). It contains **10 cells**: 1 launch FF, 8 combinational gates (`and4_1`, `a2111oi_0`, `a21oi_1`, `mux2i_1`, `xnor3_1`, two `maj3_1`, and `xor3_1`), and 1 capture FF.

In terms of the array architecture, this path traverses **exactly 8 full-adder stages**: rippling across all 4 adders in Row 1 [FA(1,0)–FA(1,3)], dropping into Row 2 at FA(2,2), and rippling across 3 adders in Row 3 [FA(3,1)–FA(3,3)], producing sum $s_3[3]$ which connects to output bit `p[6]`. The total $4 + 1 + 3 = 8$ identically matches the theoretical estimate $2N = 2 \times 4 = 8$ from Section 2.

Q3. What percentage of the total cell area is flip-flops, and why is it large?

The 16 flip-flops (`sky130_fd_sc_hd__dfrtp_1`) occupy $16 \times 25.024 = \mathbf{400.384}\,\mu m^2$ out of the total $784.502\,\mu m^2$ cell area, which constitutes **51.04 %**. This percentage is large for two reasons:

1. **Transistor Complexity.** A master-slave D flip-flop with asynchronous reset (≈24 transistors, $25.024\,\mu m^2$) is nearly 6.7× larger than a 2-input NAND gate (`nand2_1`, $3.754\,\mu m^2$).
2. **Architectural Scaling.** Registers scale linearly with operand width ($4N$), whereas the combinational array scales quadratically (N^2 ANDs and $N(N-1)$ adders). For a small $N = 4$ design, the quadratic array is compact (51 small gates), so the 16 bulky registers dominate. For wider multipliers, quadratic growth of the adder matrix rapidly reduces the flip-flop area percentage.

Q4. Why is the post-route F_{max} lower than in Part C, and which cells were added?

Post-route F_{max} dropped from **335.57 MHz** to **144.70 MHz** primarily because LibreLane signs off on *all* process corners, with the final 7.0 ns clock period dictated by the worst-case **slow corner** (`ss_100C_1v60`: 100 °C, 1.60 V), where transistor delays nearly double compared to the typical corner. (At the typical corner, post-route F_{max} is 288.04 MHz.) The remaining drop is due to physical effects that pre-layout STA ignored: real wire RC parasitics, clock-tree skew, and a 0.25 ns clock uncertainty margin.

To ensure physical manufacturability and electrical stability, the flow added **1833 physical cells** (raising the count from 67 to 1901):

- **245 tap cells** – tie N-wells to VPWR and substrate to VGND to prevent CMOS latch-up.
- **1305 decap cells** – decoupling capacitors between power rails that supply instant charge and

suppress dynamic IR drop.

- **260 fill cells** – close gaps between standard cells to maintain well/rail continuity and satisfy CMP density rules.
- **20 timing repair buffers** – inserted to fix slew rates, fanout limits, and setup/hold timing violations.
- **3 clock buffers** – high-drive `clkbuf_16` forming the clock distribution tree to deliver the clock edge with minimal skew.

Q5. What do DRC and LVS verify, and why must both pass?

DRC (Design Rule Check) verifies the physical geometry of the layout polygons against foundry manufacturing rules, checking minimum wire widths (preventing opens), track spacings (preventing shorts), via enclosures, antenna ratios, and metal densities for CMP.

LVS (Layout Versus Schematic) verifies electrical connectivity and circuit equivalence, extracting transistors and nets from layout shapes and comparing them against the synthesised netlist to confirm zero opens, zero shorts, and identical port mapping.

Both checks must pass with **0 errors** before tapeout because they guard against distinct failure modes: DRC ensures the layout is physically manufacturable without fabrication defects (otherwise the foundry rejects the GDSII file), while LVS ensures the manufactured chip functions as the intended logic circuit rather than a useless piece of silicon. Our layout achieved 0 DRC and 0 LVS errors.

Q6. How would the full adders and the critical path change for an 8-bit multiplier?

Using the architectural formulas from Section 2:

1. **Full Adder Count.** An N-bit multiplier requires $N(N-1)$ full adders. For $N = 4$: $4 \times 3 = 12$ full adders. Scaling to $N = 8$:

   $$8 \times 7 = \mathbf{56} \text{ full adders}, \quad \text{an increase of } 44 \ (\approx 4.67\times),$$

   showing quadratic $O(N^2)$ growth. Partial-product AND gates also increase from 16 to 64.

2. **Critical Path Delay.** The longest path scales with $2N$ full-adder stages. For $N = 4$: $2 \times 4 = 8$ stages (2.433 ns logic delay). For $N = 8$:

   $$2 \times 8 = \mathbf{16} \text{ stages}, \quad \approx 4.86 \text{ ns combinational delay } (\mathrm{O}(N)).$$

   Factoring in flip-flop delays, estimated $T_{\min} \approx 5.41$ ns, reducing $F_{\max}$ to $\approx$ **185** MHz (down from 335.57 MHz at $N = 4$).

Team Member Contributions

Both team members contributed equally and collaboratively across all phases of the project, including pair programming, reviewing scripts and netlists, and jointly authoring this report. The repository commit history reflects this collaborative progression:

- **Krishna Goel (2023112009)**:
  - **RTL Design & Testbench (Commits `28bcf01`, `2013210`):** Designed and implemented the structural Verilog modules (`rtl/full_adder.v` and `rtl/mult_array.v`); developed the exhaustive 256-vector self-checking testbench (`tb/tb_mult.v`) and validated the deliberate error-injection test.
  - **Manual Calculation & Waveform Verification (Commit `205c879`):** Calculated the hand-worked multiplication example ($A = 14, B = 7, P = 98$) based on his roll number and verified intermediate row products against GTKWave simulation dumps.

  - **Logic Synthesis Flow (Commit `9aed8ee`):** Configured and ran the Yosys synthesis flow (`synth/synth.tcl`) using the SkyWater 130 nm typical-corner library; verified netlist equivalence via gate-level simulation (GLS).
  - **Report Drafting:** Co-authored Sections 1 and 2, and answered architectural analysis questions Q1, Q2, and Q6.

- **Dhimant Bhuva (2023102063)**:
  - **Static Timing Analysis (Commit `42dc758`, Co-Author):** Set up and executed the OpenSTA timing analysis flow (`sta/sta.tcl`); determined worst-case slack, $T_{\min}$, and $F_{\max}$, and correlated the critical-path cell stages with the $2N = 8$ theoretical delay model.
  - **Physical Design & Signoff (Commit `3b841d1`, Co-Author):** Configured the LibreLane physical design flow (`pnr/config.json`); conducted the iterative clock period sweep (4.0 ns to 7.0 ns) across 7 runs to achieve setup timing closure in the slow process corner with 0 DRC and 0 LVS errors.
  - **Physical Verification & Layout Inspection:** Analyzed post-PnR cell breakdowns, wirelength, and slew warnings from `metrics.json`; generated and annotated layout views using KLayout.
  - **Report Authoring & Typesetting:** Authored Sections 3, 4, answered physical implementation questions Q3, Q4, and Q5, and designed and typeset the complete LaTeX report.